Ingeniería en Desarrollo de Software

Actividad | #2 |

Programa Banco Mexicano (parte 1)

Lenguajes de Programación IV

Ingeniería en Desarrollo de Software

TUTOR: Aarón Iván Salazar Macías

ALUMNO: Oscar Esteban Sánchez Leyva

FECHA: 02/Abril/2026

ÍNDICE

ÍNDICE 2

INTRODUCCIÓN 3

INVESTIGACIÓN 4

JAVA 4

PYTHON 5

C# 6

C++ 7

PHP 8

DESARROLLO 9

CÓDIGO DEL PROGRAMA (INTERFAZ GRÁFICA) 9

CONCLUSIÓN 16

REFERENCIAS 17

INTRODUCCIÓN

El propósito principal de esta actividad es fomentar el aprendizaje y desarrollo de un programa en Java utilizando los principios del paradigma de programación orientada a objetos. El programa estará diseñado para simular un sistema bancario básico que facilite a los usuarios la ejecución de operaciones esenciales, tales como realizar depósitos y retiros de manera eficiente y accesible mediante una interfaz gráfica sencilla e intuitiva.

Durante el desarrollo de esta actividad, se implementarán conceptos clave que constituyen la base de la programación orientada a objetos, tales como el diseño y uso adecuado de clases y objetos, la creación de métodos funcionales, la aplicación de estructuras de control para la toma de decisiones y la implementación de validaciones que aseguren la integridad y consistencia de las operaciones realizadas. Asimismo, se incentivará el uso práctico de herramientas como el entorno de desarrollo integrado NetBeans para organizar correctamente el código y estructurar el proyecto con claridad y eficiencia.

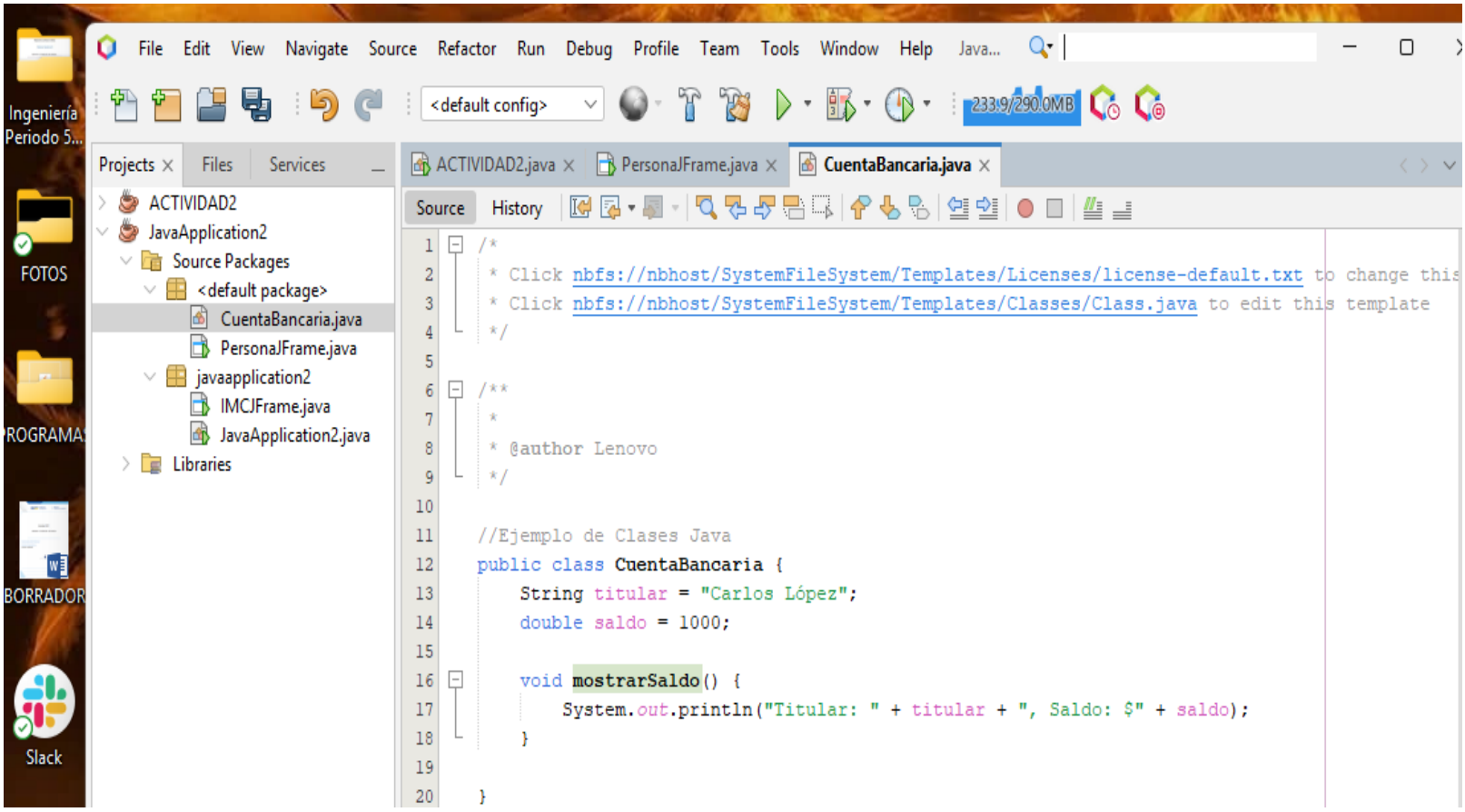
La creación de este programa no solo refuerza habilidades técnicas, sino que también resalta la relevancia de diseñar sistemas que cumplan con necesidades funcionales del mundo real. En este caso específico, se busca ofrecer una solución fundamental para gestionar operaciones bancarias básicas, como el manejo de fondos y transacciones financieras, promoviendo así un enfoque práctico en la implementación de conocimientos teóricos. Este ejercicio contribuye, además, al desarrollo del pensamiento crítico y analítico en el ámbito de la programación y el diseño de software.

INVESTIGACIÓN

A continuación, se ofrecen diversos ejemplos de clases implementadas en distintos lenguajes de programación. En cada caso, se han empleado nombres ficticios y genéricos con el objetivo de simplificar su análisis y facilitar su entendimiento para principiantes y expertos por igual. Estas representaciones pretenden destacar las similitudes y diferencias en la forma en que se estructuran las clases según el lenguaje utilizado.

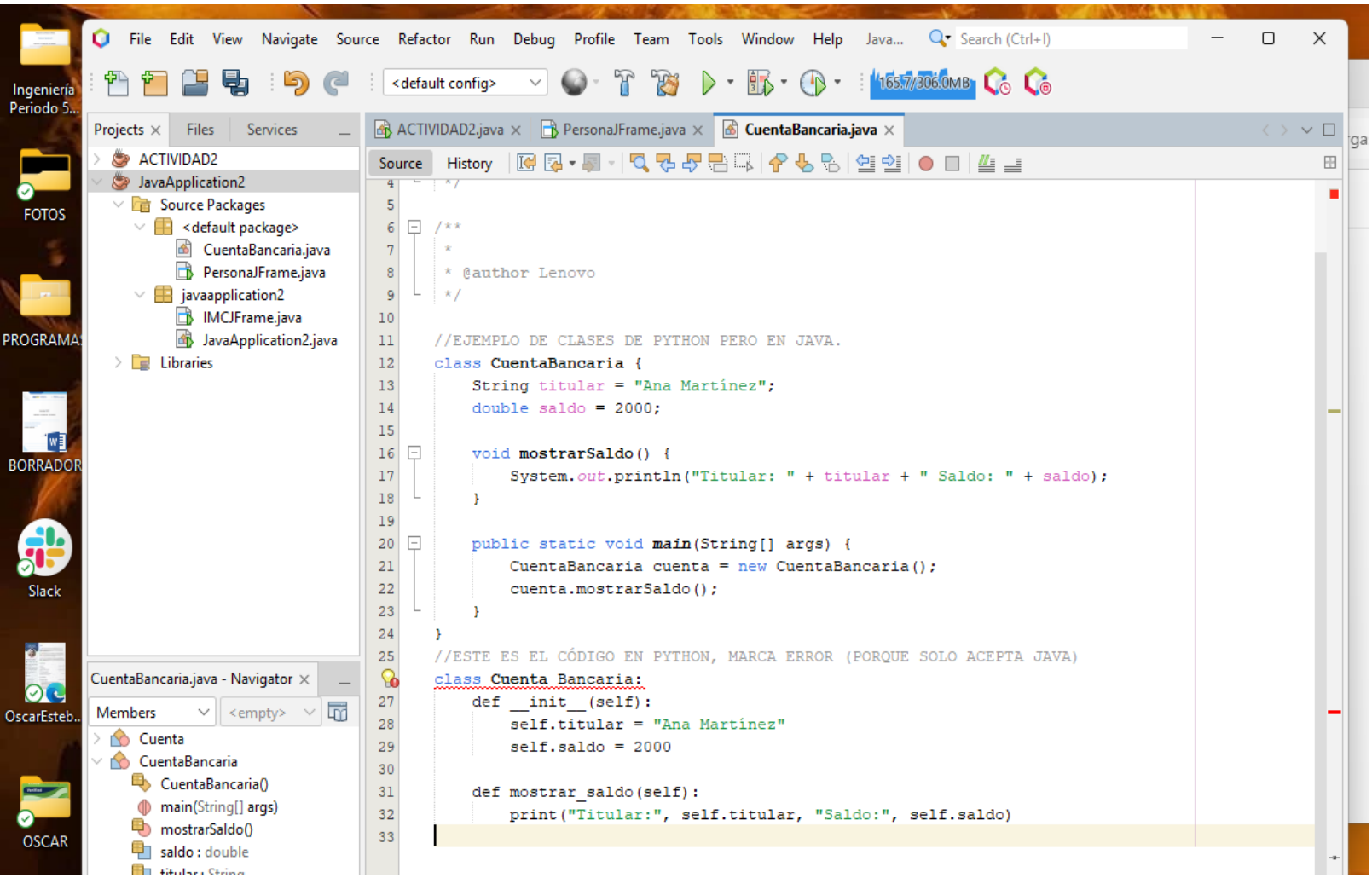
JAVA

Java es un lenguaje de programación orientado a objetos ampliamente utilizado en la creación de aplicaciones empresariales, móviles y de escritorio. Su principal ventaja radica en su portabilidad, ya que puede operar en cualquier sistema que disponga de la Máquina Virtual de Java (JVM). Además, su sintaxis estructurada facilita el trabajo con clases, métodos y objetos, optimizando el desarrollo de software.



PYTHON

Python es un lenguaje interpretado que destaca por su simplicidad y facilidad de aprendizaje. Es ampliamente empleado en áreas como el desarrollo web, la inteligencia artificial, el análisis de datos y la automatización de tareas. Su sintaxis clara y concisa facilita la escritura de menos código, haciéndolo más fácil de comprender y mantener.

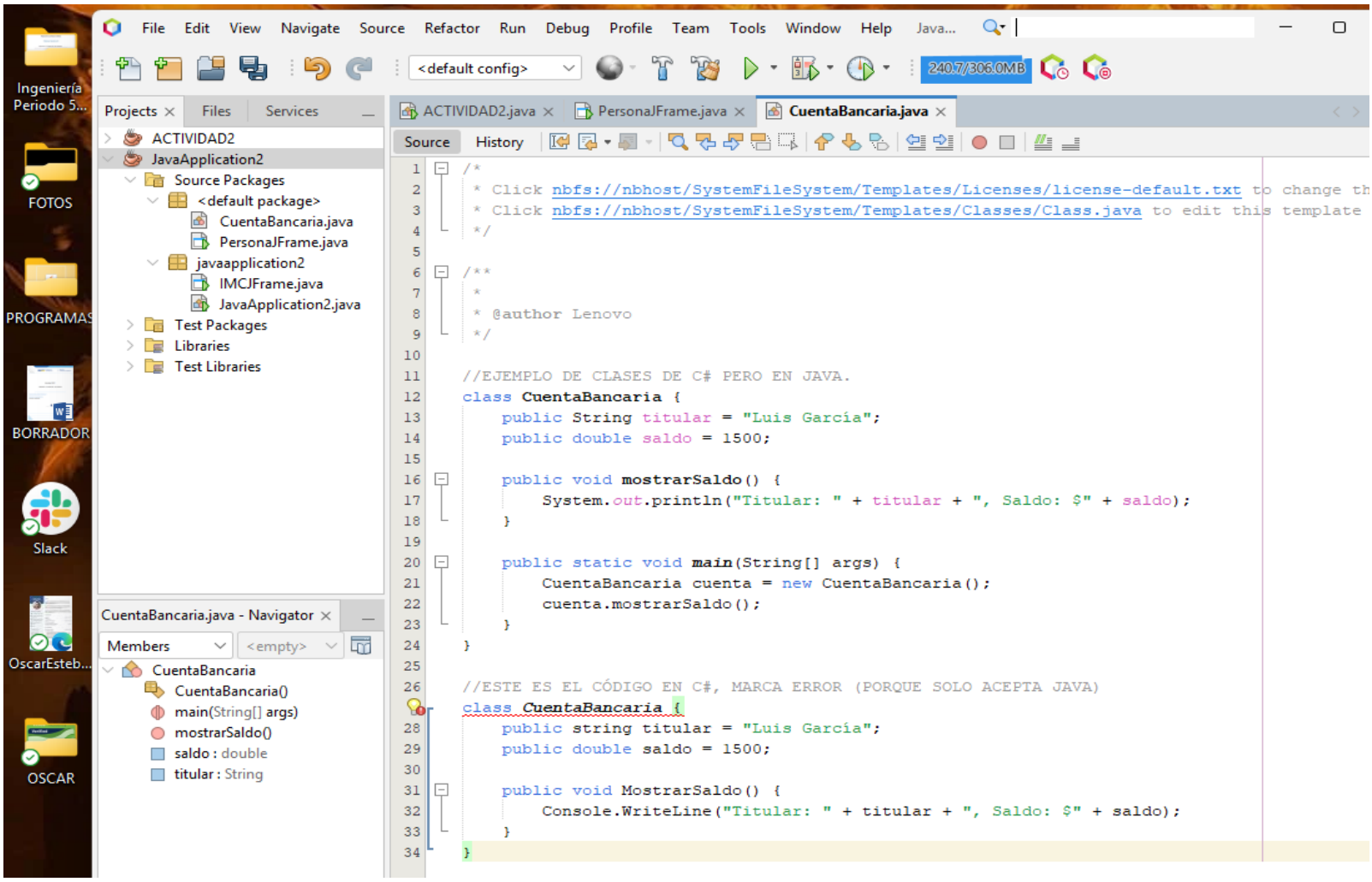


Anote ambos códigos para su ejemplo maestro, para que viera cómo quedaría en **Java** y **Python**:

- El archivo .java solo admite código escrito en Java.
- El código está en Python, por lo que genera un error.

C#

C# es un lenguaje de programación desarrollado por Microsoft, diseñado con un enfoque orientado a objetos. Se emplea principalmente para el desarrollo de aplicaciones de escritorio, aplicaciones web y videojuegos, especialmente a través del motor Unity. Su sintaxis es muy parecida a la de Java y se integra de manera impecable con la plataforma .NET.

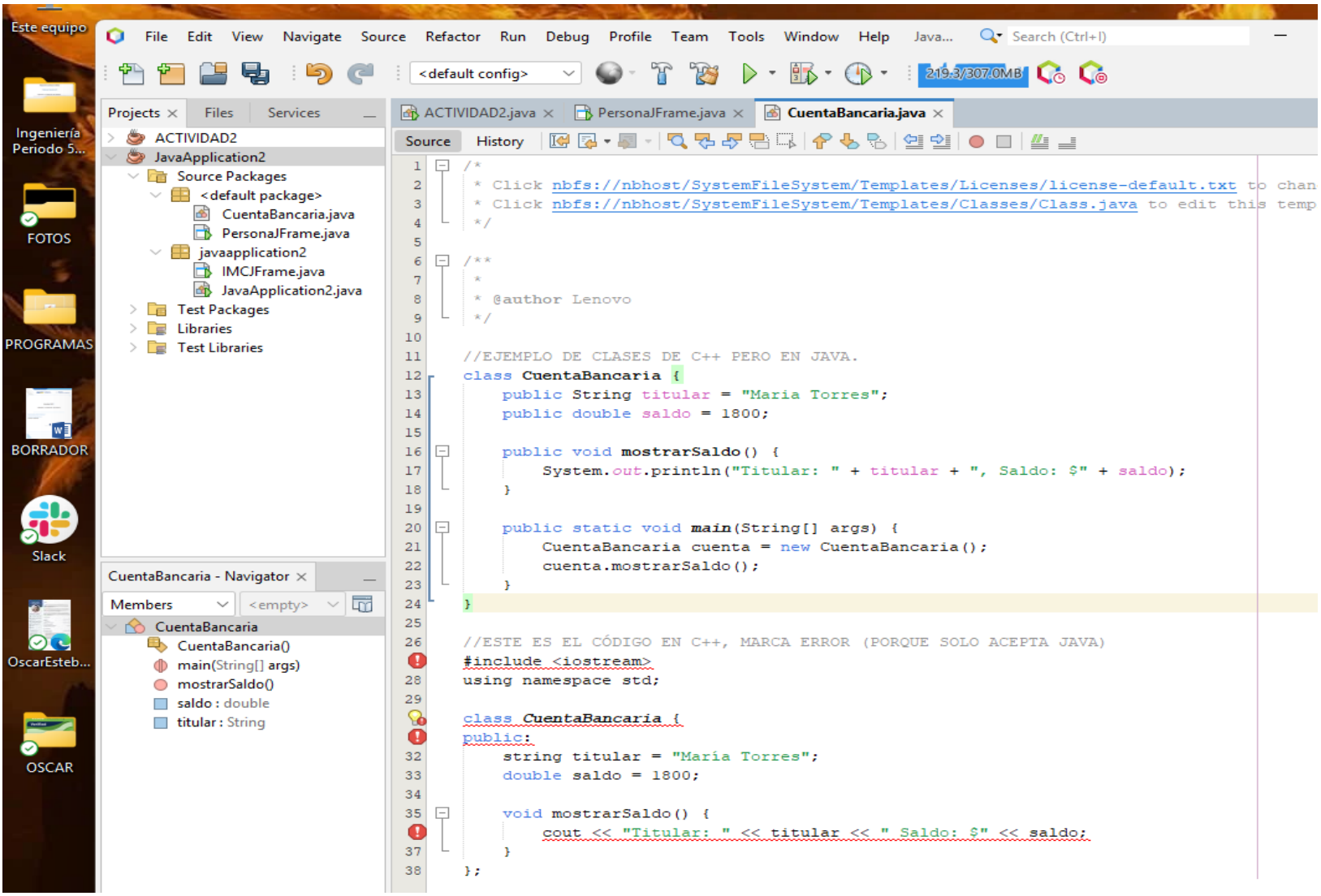


Anote ambos códigos para su ejemplo maestro, para que viera cómo quedaría en **Java** y **C#**:

- string ✕ (C#) → String ✓ (Java)
- Console.WriteLine ✕ (C#) → System.out.println ✓ (Java)

C++

C++ es un potente lenguaje de programación que combina la flexibilidad de la programación orientada a objetos con las características de la programación estructurada. Su uso es muy común en el desarrollo de sistemas operativos, videojuegos y aplicaciones que demandan un alto nivel de eficiencia.

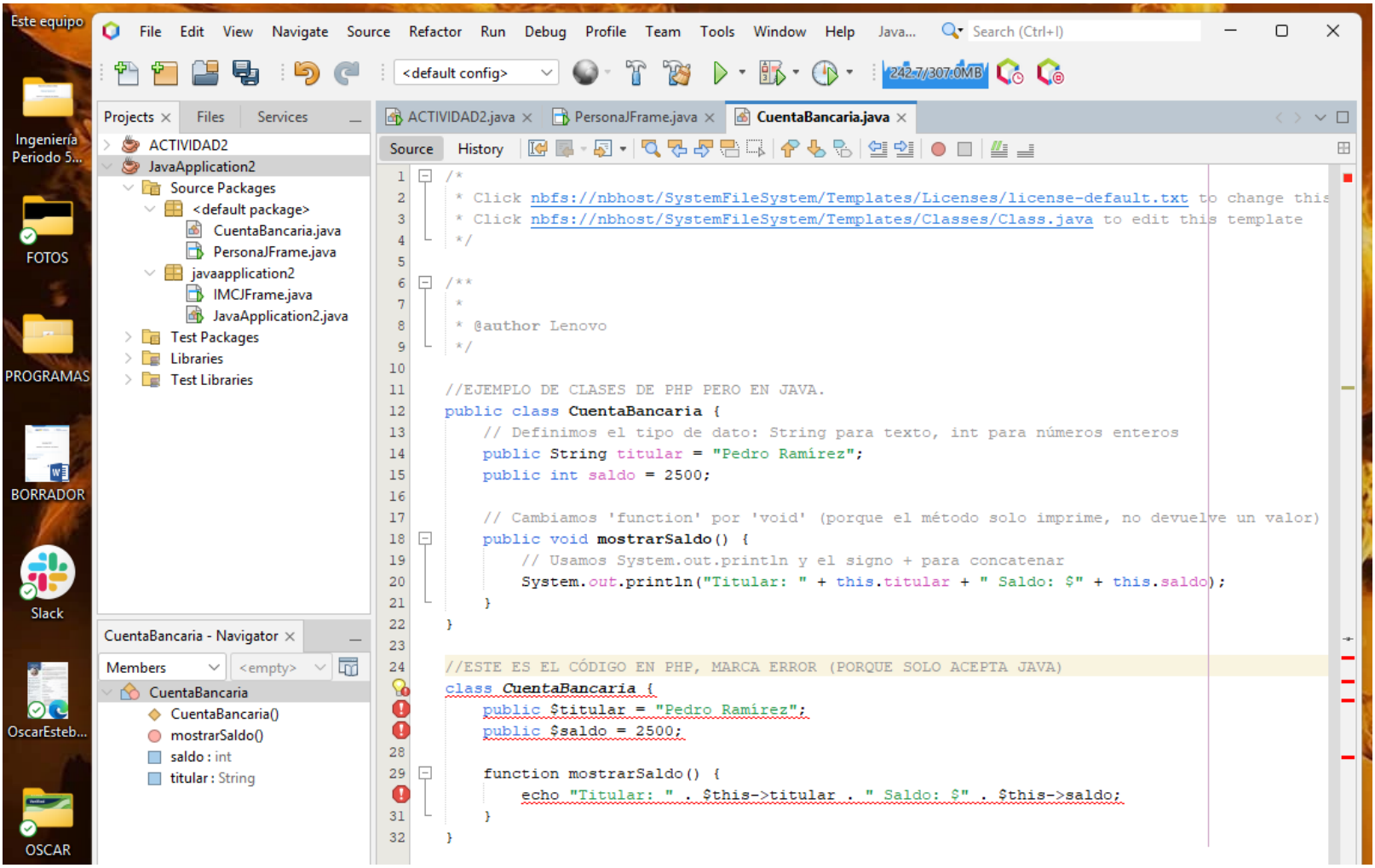


Anoté ambos códigos para su ejemplo maestro, para que viera cómo quedaría en **Java** y **C++**, por eso marca error, porque Java no reconoce:

- **1.-** (#include) **2.-** (using namespace std;) **3.-** (cout <<)

PHP

PHP es un lenguaje de programación orientado al desarrollo web, ampliamente utilizado para crear páginas dinámicas que interactúan con bases de datos. Su integración con HTML es sencilla, lo que lo convierte en una opción muy popular en servidores web.



Anoté ambos códigos para su ejemplo maestro, para que viera cómo quedaría en **Java** y **PHP**; marca error, por lo siguiente:

- **En PHP:** Usas -> para acceder a propiedades y . para unir (concatenar) textos.
- **En Java:** Usas un punto . para acceder a propiedades y el signo + para unir textos. También se usa System.out.println para imprimir en consola en lugar de echo.

DESARROLLO

CÓDIGO DEL PROGRAMA (INTERFAZ GRÁFICA)

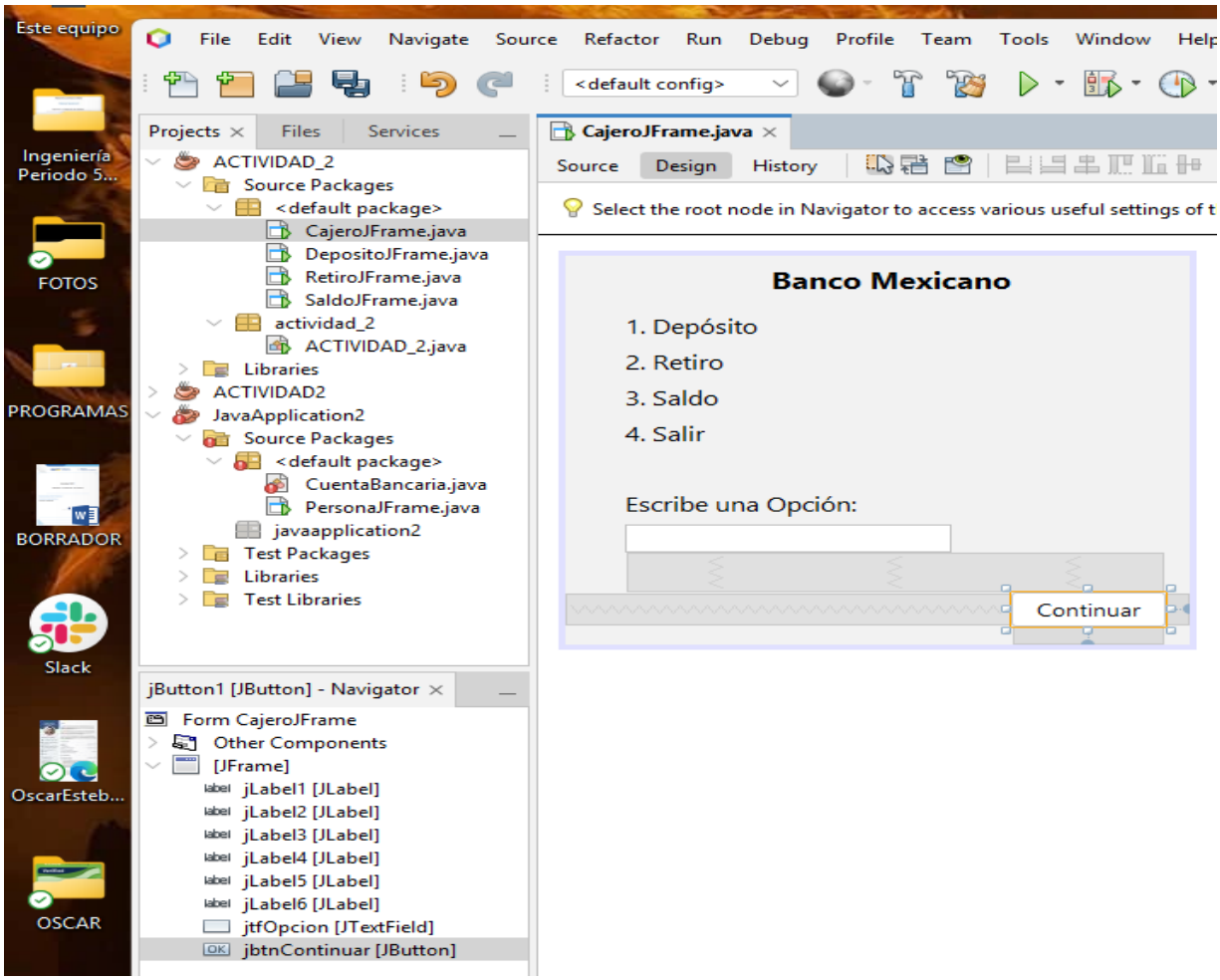
Primero, describo el contexto de la actividad, seguido de la incorporación de evidencia visual. Se desarrolló un sistema bancario básico en Java utilizando programación orientada a objetos y una interfaz gráfica con Swing para interacción mediante botones y campos de texto.

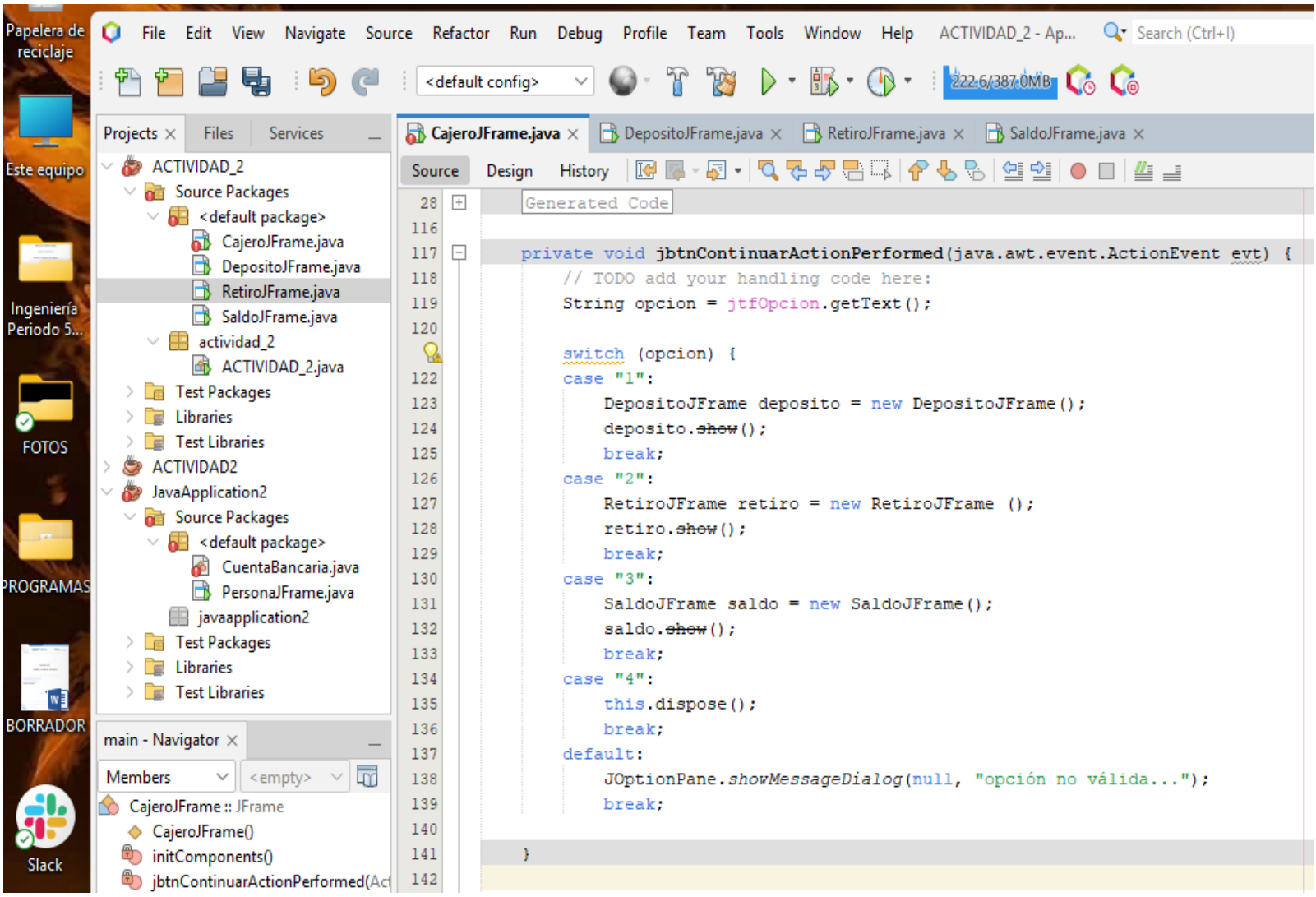
La interfaz principal permite ingresar una cantidad de dinero y cuenta con botones para "Depositar" y "Retirar". Además, incluye una etiqueta que muestra el saldo actual, que se actualiza automáticamente tras cada operación.

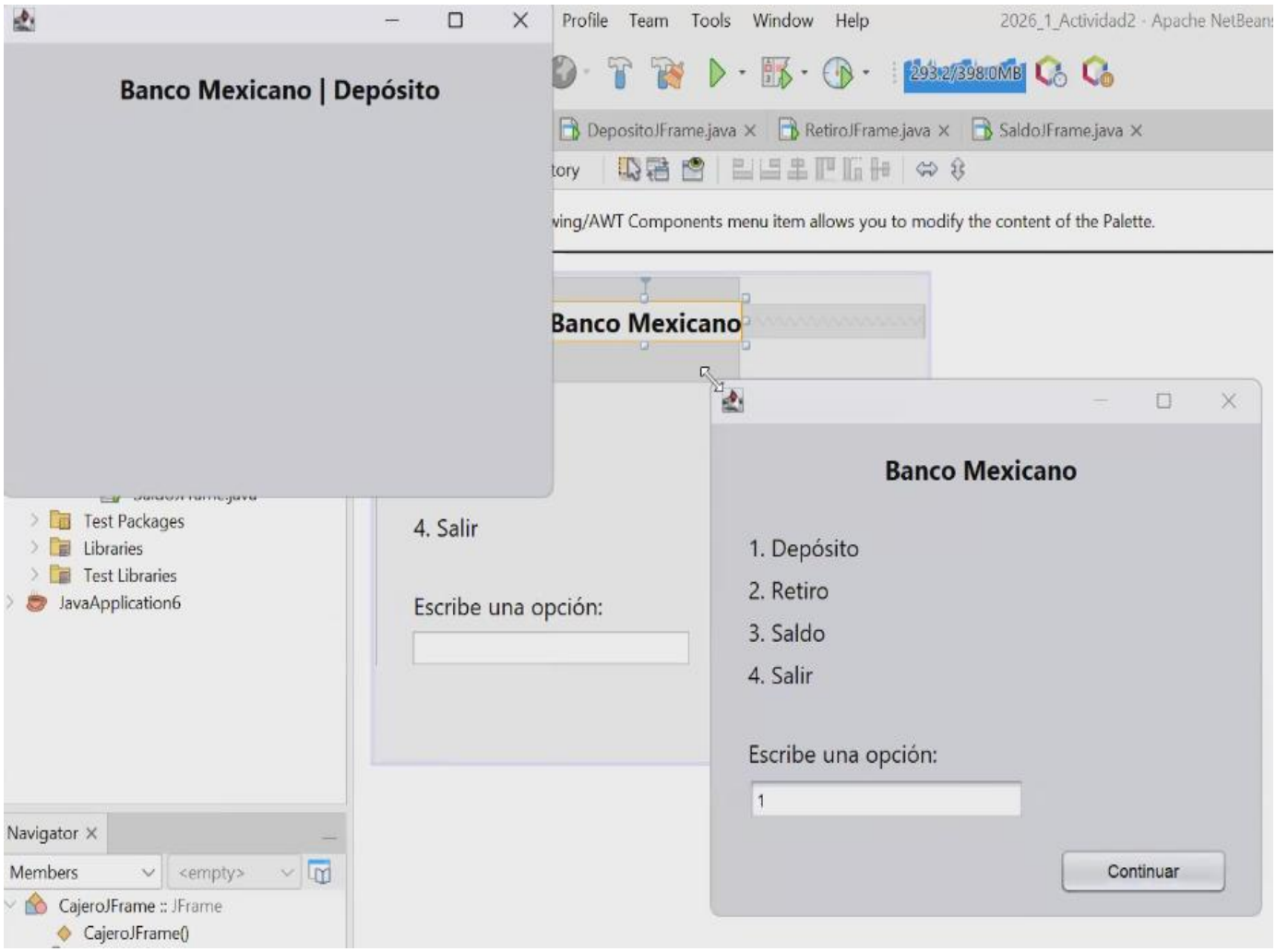
En la función de depósito, se pueden ingresar cantidades positivas que se suman al saldo. Si el valor es inválido, se muestra un error. En el caso del retiro, se verifica que haya saldo suficiente; de lo contrario, aparece una alerta de saldo insuficiente.

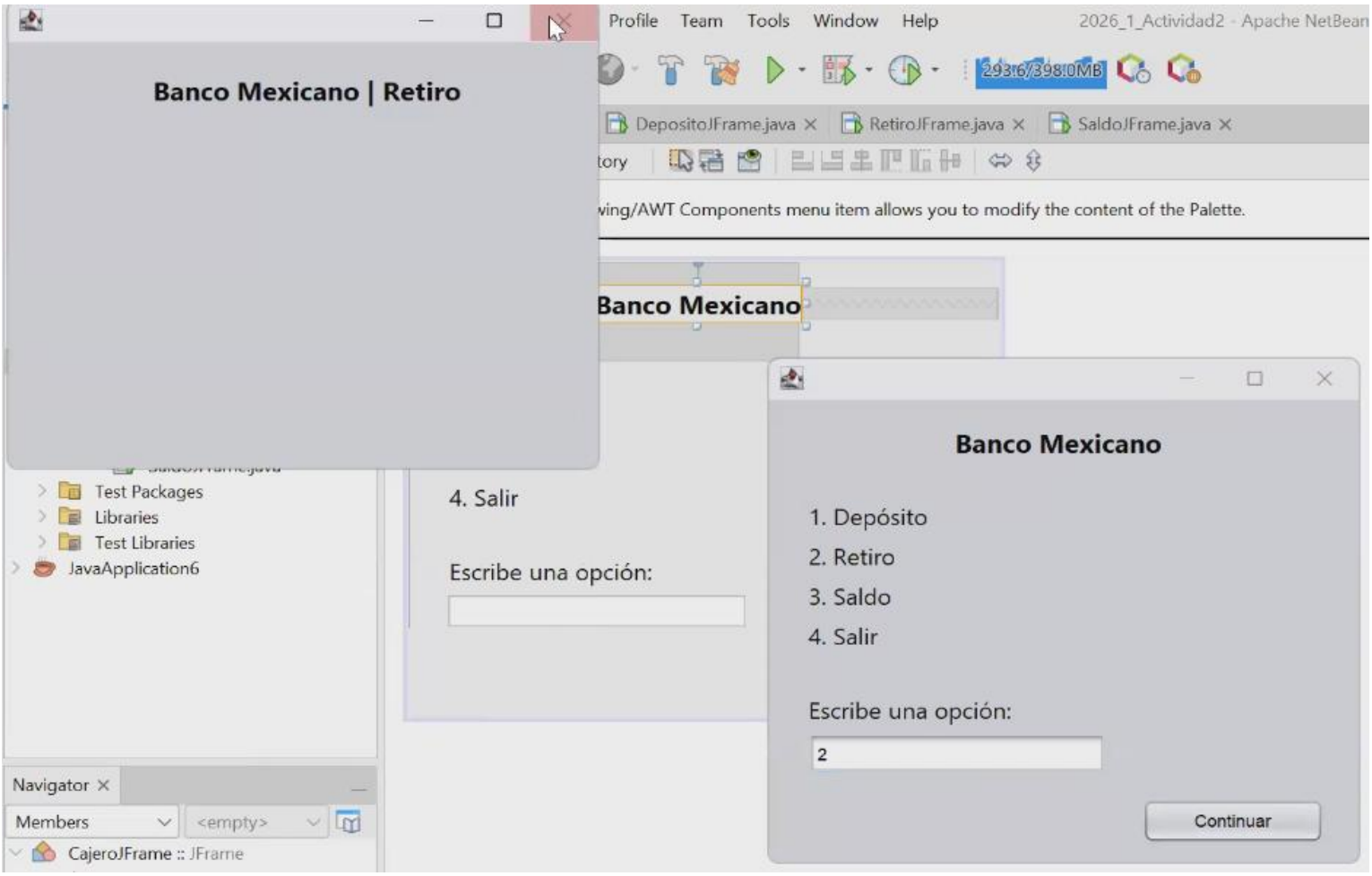
Para su implementación, se emplearon condicionales (if) para validar datos y eventos (ActionListener) para ejecutar acciones al interactuar con los botones. También se utilizaron conceptos como clases, atributos y métodos.

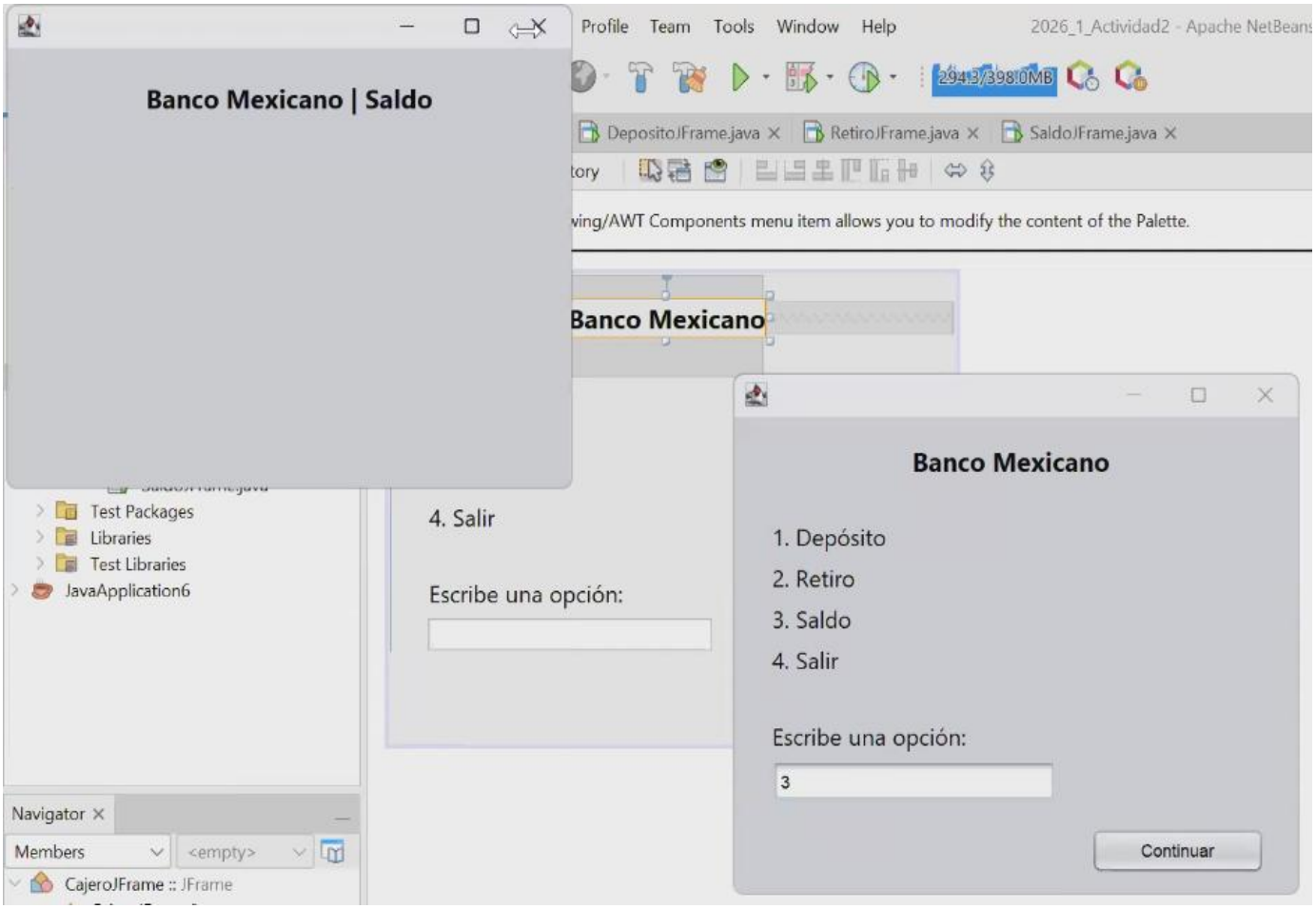
Finalmente, se realizaron pruebas para garantizar que el sistema operara correctamente, tanto en depósitos como en retiros, asegurando la actualización precisa del saldo.


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Projects Files Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Libraries
ACTIVIDAD2
JavaApplication2
Source Packages
<default package>
CuentaBancaria.java
PersonaJFrame.java
javaapplication2
Test Packages
Libraries
Test Libraries
jButton1 [JButton] - Navigator
Form CajeroJFrame
Other Components
[JFrame]
jLabel1 [JLabel]
jLabel2 [JLabel]
jLabel3 [JLabel]
jLabel4 [JLabel]
jLabel5 [JLabel]
jLabel6 [JLabel]
jtfOpcion [JTextField]
jbtnContinuar [JButton]
CajeroJFrame.java
Source Design History
Select the root node in Navigator to access various useful settings of t
Banco Mexicano
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una Opción:
Continuar

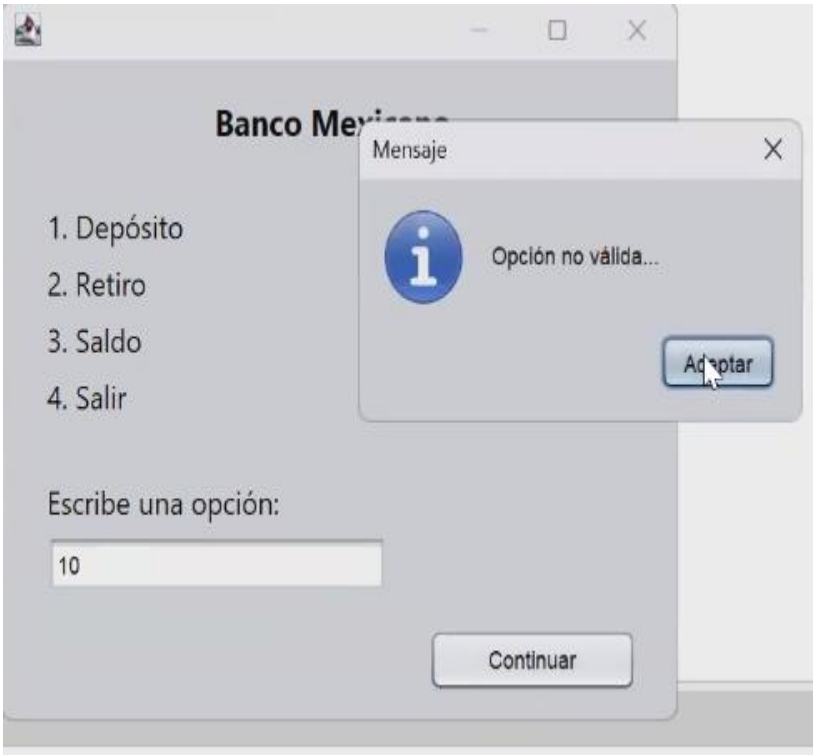

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
ACTIVIDAD_2 - Ap...
Search (Ctrl+I)
<default config>
222.6/387.0MB
Projects Files Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
Test Libraries
ACTIVIDAD2
JavaApplication2
Source Packages
<default package>
CuentaBancaria.java
PersonaJFrame.java
javaapplication2
Test Packages
Libraries
Test Libraries
main - Navigator
Members
<empty>
CajeroJFrame :: JFrame
CajeroJFrame()
initComponents()
jbtnContinuarActionPerformed(Act
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
Source Design History
Generated Code
private void jbtnContinuarActionPerformed(java.awt.event.ActionEvent evt) {
// TODO add your handling code here:
String opcion = jtfOpcion.getText();
switch (opcion) {
case "1":
DepositoJFrame deposito = new DepositoJFrame();
deposito.show();
break;
case "2":
RetiroJFrame retiro = new RetiroJFrame ();
retiro.show();
break;
case "3":
SaldoJFrame saldo = new SaldoJFrame();
saldo.show();
break;
case "4":
this.dispose();
break;
default:
JOptionPane.showMessageDialog(null, "opción no válida...");
break;
}

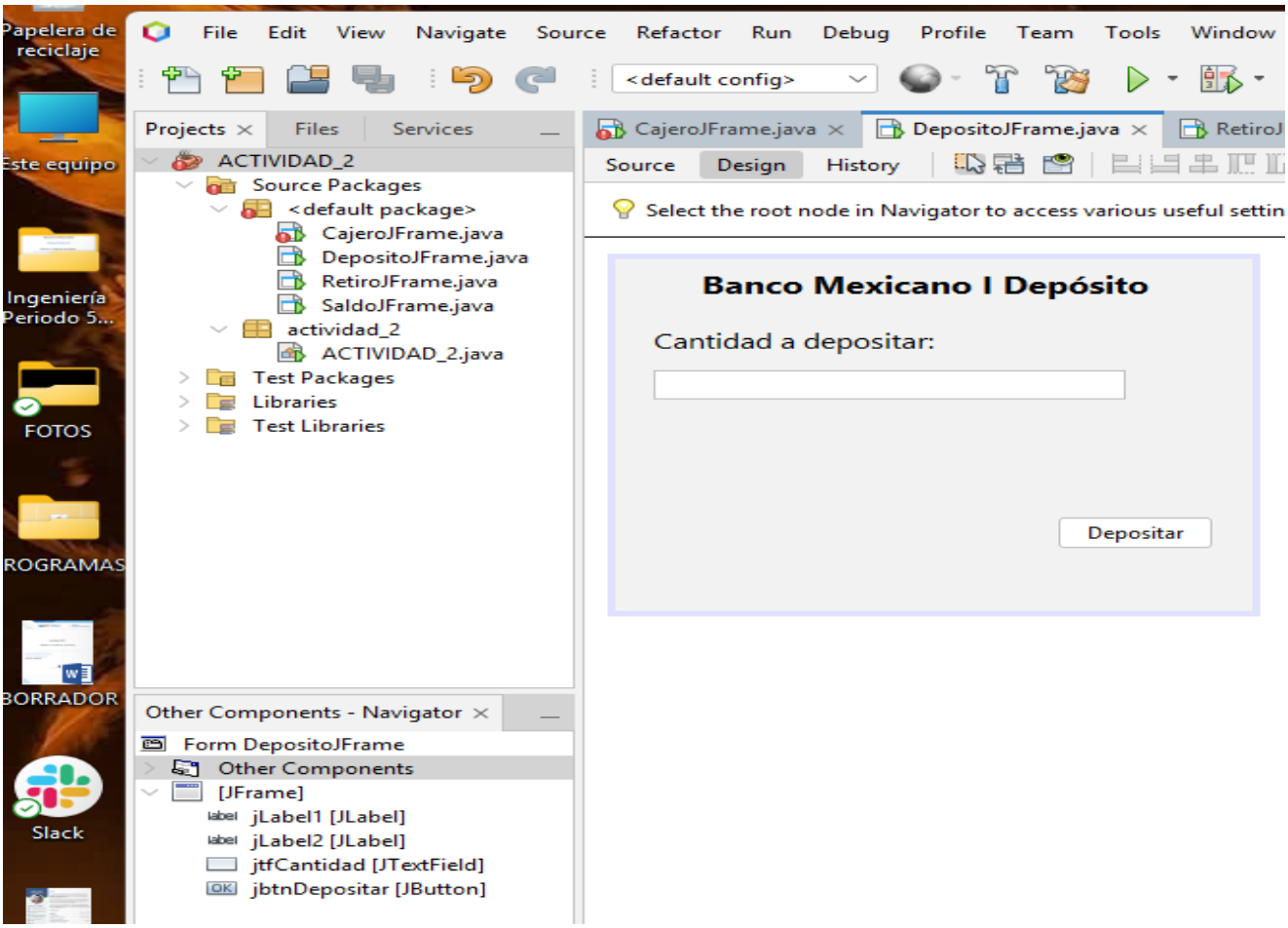

Banco Mexicano | Depósito
Profile Team Tools Window Help
2026_1_Actividad2 - Apache NetBeans
293.2/398.0MB
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
ving/AWT Components menu item allows you to modify the content of the Palette.
Banco Mexicano
Test Packages
Libraries
Test Libraries
JavaApplication6
4. Salir
Escribe una opción:
Banco Mexicano
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una opción:
1
Continuar
Navigator
Members
<empty>
CajeroJFrame :: JFrame
CajeroJFrame()

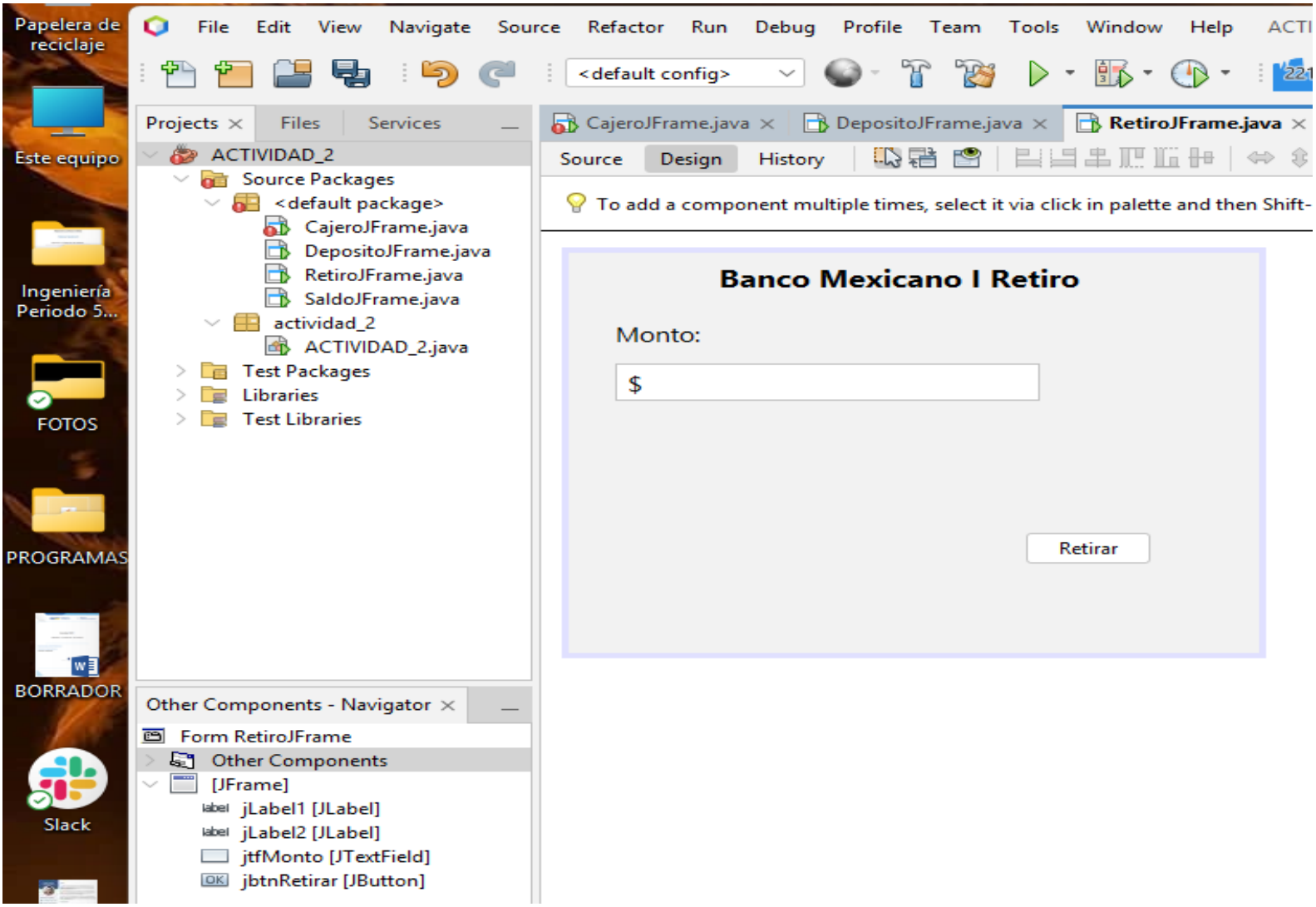

Banco Mexicano | Retiro
Profile Team Tools Window Help
2026_1_Actividad2 - Apache NetBeans
293.6/398.0MB
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
ving/AWT Components menu item allows you to modify the content of the Palette.
Banco Mexicano
Test Packages
Libraries
Test Libraries
JavaApplication6
4. Salir
Escribe una opción:
Banco Mexicano
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una opción:
2
Continuar
Navigator
Members
<empty>
CajeroJFrame :: JFrame

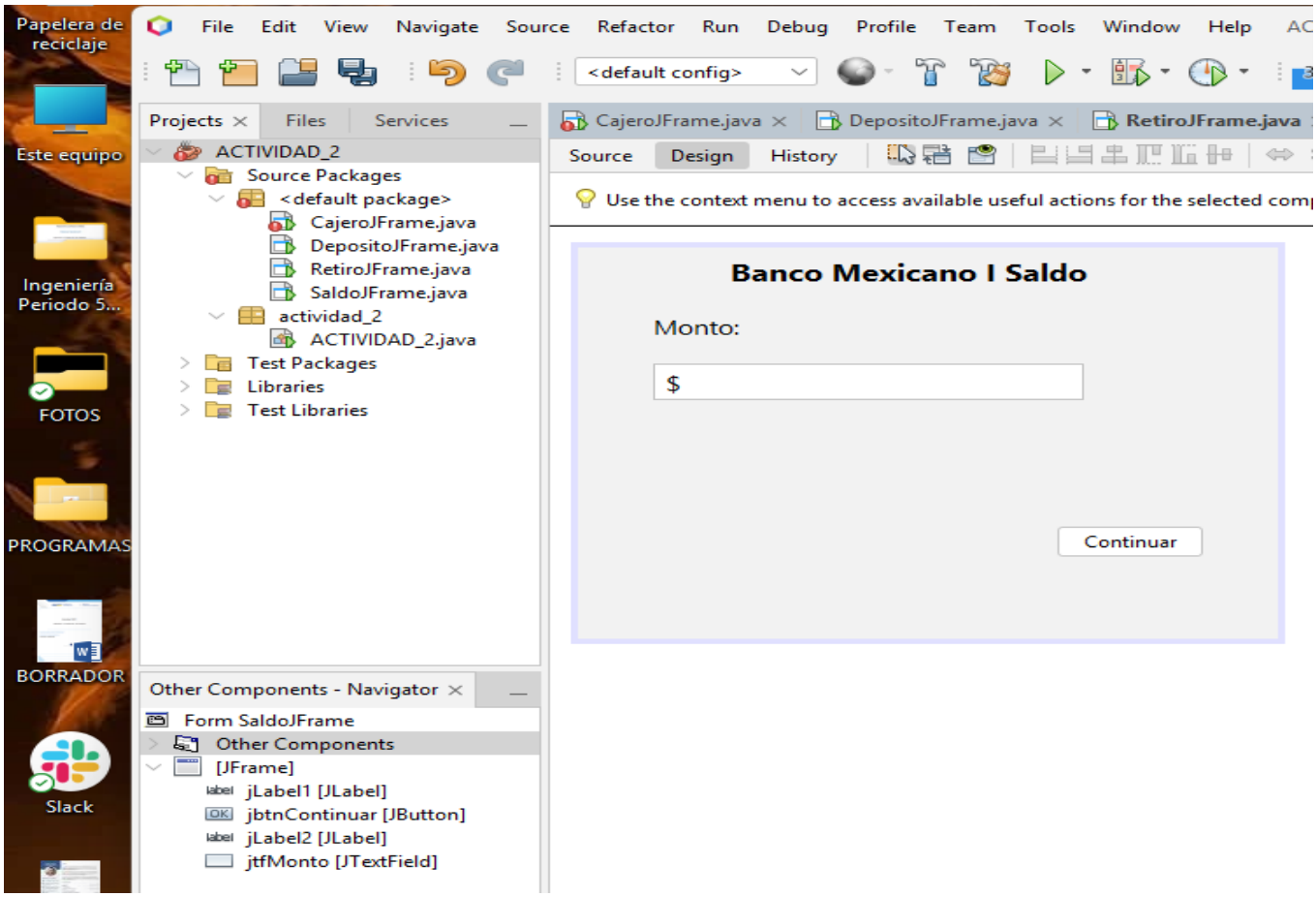

Banco Mexicano | Saldo
Profile Team Tools Window Help
2026_1_Actividad2 - Apache NetBeans
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
wing/AWT Components menu item allows you to modify the content of the Palette.
Banco Mexicano
Test Packages
Libraries
Test Libraries
JavaApplication6
4. Salir
Escribe una opción:
Banco Mexicano
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una opción:
3
Continuar
Navigator
Members
CajeroJFrame :: JFrame

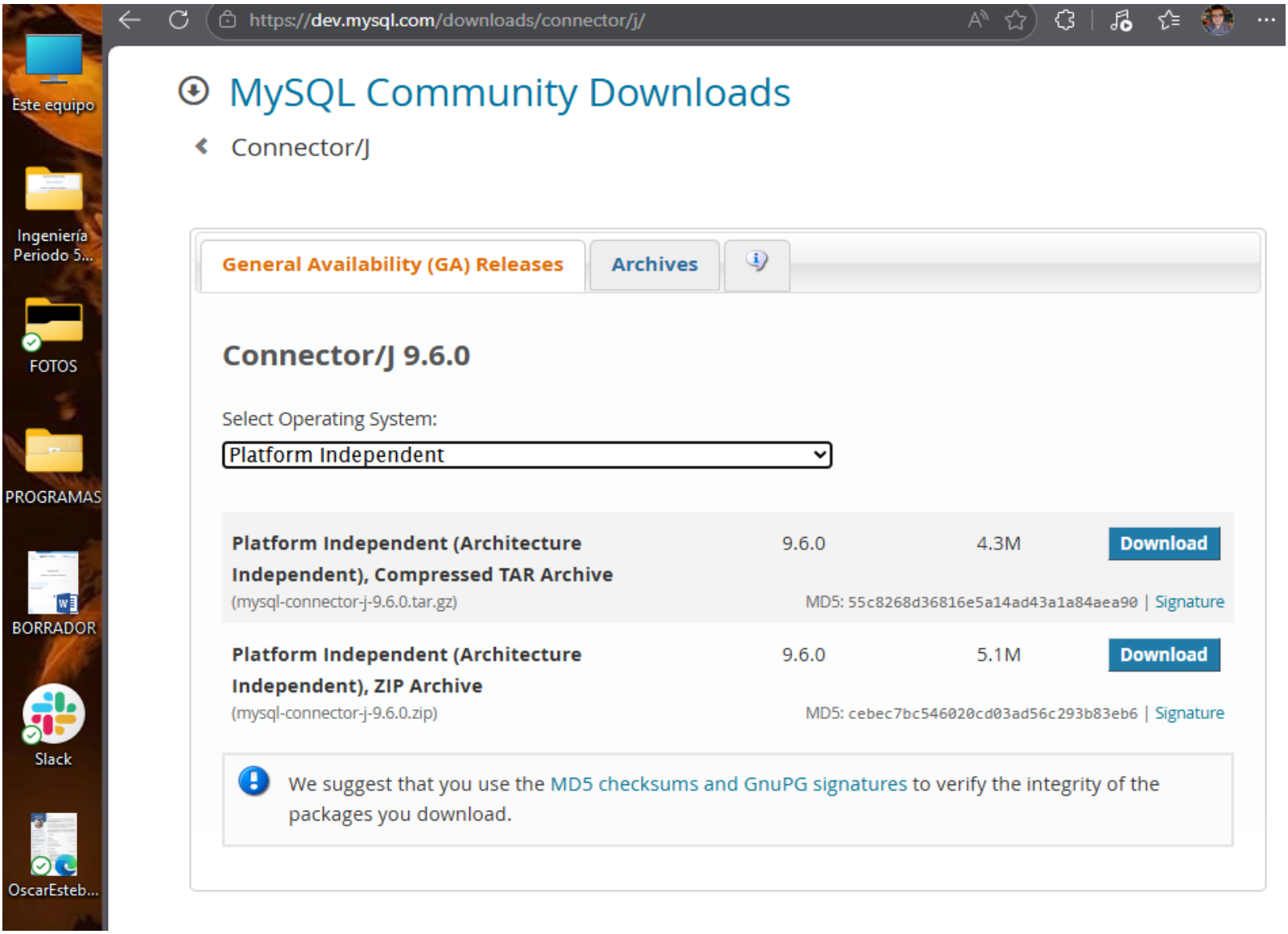
Banco Mexicano
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una opción:
4
Continuar

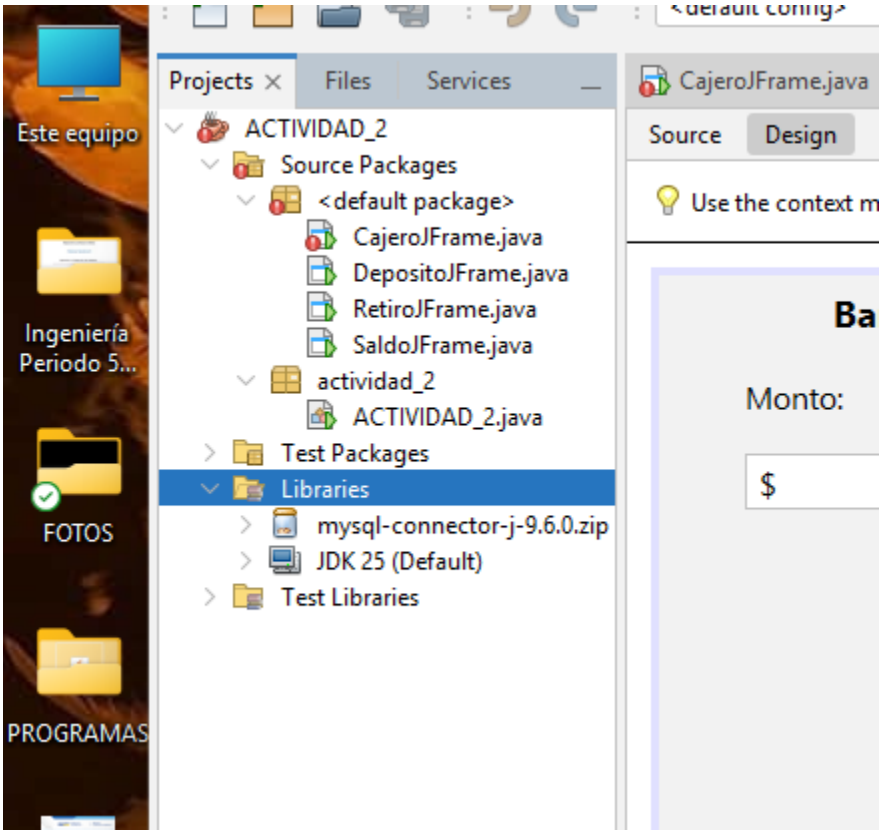

Mensaje
Opción no válida...
1. Depósito
2. Retiro
3. Saldo
4. Salir
Escribe una opción:
10
Continuar

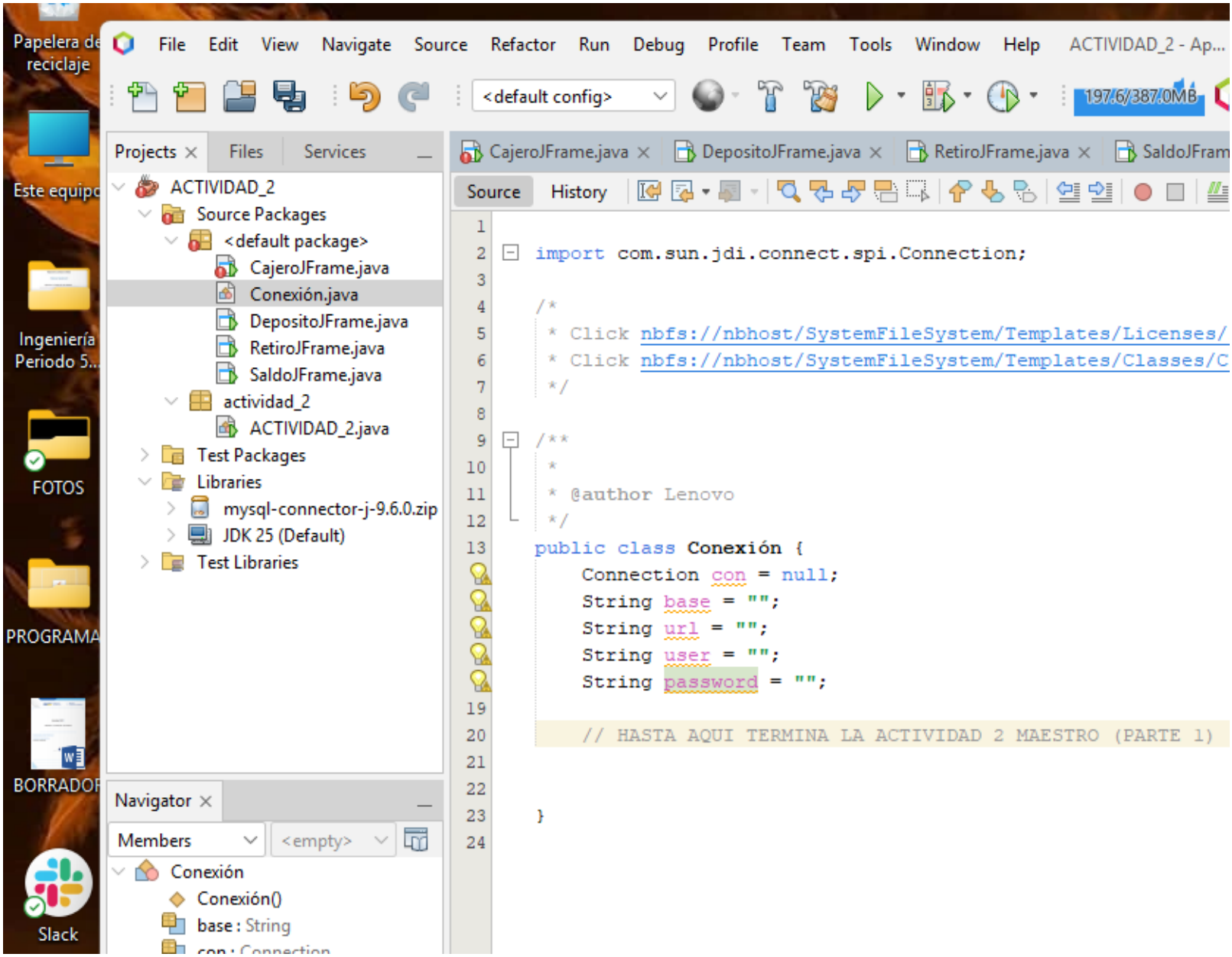

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window
<default config>
Projects Files Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
Test Libraries
Other Components - Navigator
Form DepositoJFrame
Other Components
[JFrame]
jLabel1 [JLabel]
jLabel2 [JLabel]
jtfCantidad [JTextField]
jbtnDepositar [JButton]
CajeroJFrame.java
DepositoJFrame.java
Source Design History
Select the root node in Navigator to access various useful settin
Banco Mexicano I Depósito
Cantidad a depositar:
Depositar


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Projects Files Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
Test Libraries
Other Components - Navigator
Form RetiroJFrame
Other Components
[JFrame]
jLabel1 [JLabel]
jLabel2 [JLabel]
jtfMonto [JTextField]
jbtnRetirar [JButton]
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
Source Design History
To add a component multiple times, select it via click in palette and then Shift-
Banco Mexicano I Retiro
Monto:
$
Retirar


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Projects Files Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
Test Libraries
Other Components - Navigator
Form SaldoJFrame
Other Components
[JFrame]
jLabel1 [JLabel]
jbtnContinuar [JButton]
jLabel2 [JLabel]
jtfMonto [JTextField]
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
Source Design History
Use the context menu to access available useful actions for the selected com
Banco Mexicano I Saldo
Monto:
$
Continuar


https://dev.mysql.com/downloads/connector/j/
MySQL Community Downloads
Connector/J
General Availability (GA) Releases
Archives
Connector/J 9.6.0
Select Operating System:
Platform Independent
Platform Independent (Architecture Independent), Compressed TAR Archive
9.6.0
4.3M
Download
(mysql-connector-j-9.6.0.tar.gz)
MD5: 55c8268d36816e5a14ad43a1a84aea90 | Signature
Platform Independent (Architecture Independent), ZIP Archive
9.6.0
5.1M
Download
(mysql-connector-j-9.6.0.zip)
MD5: cebec7bc546020cd03ad56c293b83eb6 | Signature
We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.


Projects
Files
Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
mysql-connector-j-9.6.0.zip
JDK 25 (Default)
Test Libraries
CajeroJFrame.java
Source
Design
Use the context m
Ba
Monto:
$


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
ACTIVIDAD_2 - Ap...
<default config>
197.6/387.0MB
Projects
Files
Services
ACTIVIDAD_2
Source Packages
<default package>
CajeroJFrame.java
Conexión.java
DepositoJFrame.java
RetiroJFrame.java
SaldoJFrame.java
actividad_2
ACTIVIDAD_2.java
Test Packages
Libraries
mysql-connector-j-9.6.0.zip
JDK 25 (Default)
Test Libraries
CajeroJFrame.java
DepositoJFrame.java
RetiroJFrame.java
Source
History
import com.sun.jdi.connect.spi.Connection;
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/C
*/
/**
*
* @author Lenovo
*/
public class Conexión {
Connection con = null;
String base = "";
String url = "";
String user = "";
String password = "";
// HASTA AQUI TERMINA LA ACTIVIDAD 2 MAESTRO (PARTE 1)
}
Navigator
Members
<empty>
Conexión
Conexión()
base : String

CONCLUSIÓN

En el transcurso de esta actividad, se llevó a cabo el desarrollo de un programa funcional utilizando el lenguaje de programación Java, implementando el paradigma de programación orientada a objetos. Este enfoque no solo permitió entender mejor la importancia de establecer una estructura adecuada en el diseño de un sistema, sino que también sentó las bases para la organización y modularización del código. Una de las características destacadas fue la incorporación de una interfaz gráfica que facilitó considerablemente la interacción del usuario con la aplicación, permitiendo realizar operaciones básicas —como depósitos y retiros— de manera clara, ágil y accesible para cualquier tipo de usuario.

Además, esta práctica ofreció una valiosa oportunidad para reforzar conocimientos fundamentales en programación, tales como el manejo adecuado de estructuras condicionales, la gestión de eventos y la aplicación eficiente de clases en un entorno práctico. Dichos elementos resultaron esenciales para enfrentar problemas comunes dentro del ámbito de la programación y fortalecer las competencias de análisis y lógica necesarias para la resolución eficaz de desafíos técnicos.

Cabe destacar que esta experiencia no solo contribuyó al aprendizaje teórico-práctico, sino que también ayudó a trasladar conceptos aprendidos en un entorno académico a situaciones hipotéticas más cercanas al ámbito profesional. En este sentido, fue posible construir un puente entre los principios teóricos y su aplicación en posibles escenarios reales, promoviendo la mejora continua en habilidades clave para el desarrollo de software.

En última instancia, esta actividad representó un avance significativo en la formación como programador, ya que estableció las bases necesarias para emprender el diseño y la creación de sistemas más complejos.

REFERENCIAS

Monterde, U. M. (s. f.). *Lenguajes de programación*. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/2655/mod_resource/content/1/UAPA-Lenguajes-Programacion/index.html

Morais, R. (2023, agosto 24). *¿Qué son las interfaces en programación? Ejemplos de Java y Golang*. Medium. https://medium.com/@fuenrob/qu%C3%A9-son-las-interfaces-en-programaci%C3%B3n-ejemplos-de-java-y-golang-98296235417f

appleute. (2024, 8 noviembre). *Interfaz de programación: ¿Qué es?* Appleute. https://www.appleute.de/es/biblioteca-para-desarrolladores-de-aplicaciones/interfaz-de-programacion/

Chakray, & Chakray. (2025, 24 febrero). *Lenguajes de programación: tipos y características*. Chakray. https://chakray.com/es/lenguajes-programacion-tipos-caracteristicas/